

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC MÁY TÍNH



SELF-STUDY

Đề tài: YOLO (Theory)

Mentor: My self
Student: Lại Khánh Hoàng

Tp. Hồ Chí Minh



Mục lục

1	Tổng quan	2
1.1	Concept of YOLO	2
1.2	Anchor boxes	2
1.3	Evaluation metric	3
1.4	Non-max Suppression	3
2	In Depth	4
2.1	Kiến trúc YOLOv3	4
2.2	Output của anchor boxes	5
2.3	Why Sigmoid ?	5
3	Loss function	6
3.1	YOLO-v2 Loss	6
3.2	YOLO-v3 Loss - so với YOLO-v2	6

1 Tổng quan

1.1 Concept of YOLO

- Là model dùng cho bài toán Object Detection, rất nhanh và có thể chạy real-time.
- YOLO chia ảnh đầu vào thành một lưới $S \times S$, mỗi ô chỉ dự đoán **một** vật thể.
- Với mỗi ô, output đầu ra dùng để train là:

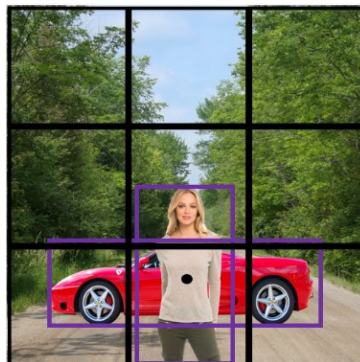
$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Trong đó:

- p_c : 0 hoặc 1, đại diện cho việc ô có chứa vật thể hay không
- b_x : tâm của vật thể theo trục x, giá trị trong khoảng (0,1)
- b_y : tâm của vật thể theo trục y, giá trị trong khoảng (0,1)
- b_h : chiều cao của vật thể, giá trị có thể > 1
- b_w : chiều rộng của vật thể, giá trị có thể > 1
- c_1, c_2, c_3 : class labels (one-hot vecto)

1.2 Anchor boxes

Việc chỉ dùng grid cells là có khuyết điểm. Giả sử trong trường hợp có nhiều vật thể trong ô, ví dụ trong ảnh có cô gái và chiếc xe, ta phải chọn ô tô hay cô gái ? Để giải quyết điều này người ta dùng anchor box.



Lúc này mỗi cell có 2 anchor boxes, mỗi anchor box đóng vai trò như một container. Khi đó output sẽ mở rộng ra để chứa thông tin của 2 anchor boxes: $(N \times N) \times [\text{num_anchors} \times (5 + \text{num_classes})]$
Ví dụ:

$$y = \begin{bmatrix} p_{c1} \\ b_{x1} \\ b_{y1} \\ b_{h1} \\ b_{w1} \\ \dots \\ c_{12} \\ c_{22} \\ c_{32} \end{bmatrix}$$



1.3 Evaluation metric

$$\text{IoU} = \frac{\text{Area of overlap}}{\text{Area of union}}$$

1.4 Non-max Suppression

Non-max Suppression là một thuật toán quen thuộc dùng để dọn dẹp khi có nhiều ô đưa ra cùng 1 vật thể. Quá trình bao gồm 3 bước:

- Bỏ qua tất cả các box có class score ≤ 0.6 .
- Chọn box có class score lớn nhất làm đầu ra.
- Bỏ các box có IoU lớn hơn hoặc bằng 0.5.

Trong đó:

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_2 \end{bmatrix} \rightarrow \text{Class Score} = \begin{bmatrix} p_c \times c_1 \\ p_c \times c_2 \\ p_c \times c_3 \end{bmatrix}$$



2 In Depth

2.1 Kiến trúc YOLOv3

YOLOv3 là một mô hình phát hiện vật thể **một giai đoạn** (single-stage), thực hiện đồng thời hai nhiệm vụ: xác định vị trí (localization) và phân loại (classification) các vật thể trong ảnh. Mô hình bao gồm hai phần chính: **mạng trích chọn đặc trưng (backbone)** và **các tầng dự đoán (detection heads)**, cụ thể:

- Input: $(B, 3, H, W)$, với $H = W = 416$ trong cấu hình mặc định của YOLOv3.
- Sử dụng Darknet-53 để làm feature extractor:

1. **Conv + BN + LeakyReLU:** $(B, 3, H, W) \rightarrow (B, 32, \frac{H}{2}, \frac{W}{2})$

2. **Residual Block 1** gồm 1 residual block:

$$(B, 32, \frac{H}{2}, \frac{W}{2}) \rightarrow (B, 64, \frac{H}{4}, \frac{W}{4})$$

3. **Residual Block 2** gồm 2 residual blocks:

$$(B, 64, \frac{H}{4}, \frac{W}{4}) \rightarrow (B, 128, \frac{H}{8}, \frac{W}{8})$$

(**Route 1**) được trích từ stage này với kích thước $(B, 256, 52, 52)$ nếu $H = W = 416$.

4. **Residual Block 3** gồm 8 residual blocks:

$$(B, 128, \frac{H}{8}, \frac{W}{8}) \rightarrow (B, 256, \frac{H}{16}, \frac{W}{16})$$

(**Route 2**) được trích tại đây với kích thước $(B, 512, 26, 26)$.

5. **Residual Block 4** gồm 8 residual blocks:

$$(B, 256, \frac{H}{16}, \frac{W}{16}) \rightarrow (B, 512, \frac{H}{32}, \frac{W}{32})$$

6. **Residual Block 5** gồm 4 residual blocks:

$$(B, 512, \frac{H}{32}, \frac{W}{32}) \rightarrow (B, 1024, \frac{H}{32}, \frac{W}{32})$$

(**Route 3**) được trích tại đây với kích thước $(B, 1024, 13, 13)$.

Tóm lại, output của Darknet-53 là gồm 3 layer để lần lượt trích xuất các vật thể nhỏ, vừa và lớn.

Listing 1: YOLOv3 detection heads

```
self.yolo_layers = nn.ModuleList([
    YOLOLayer(anchors[0], num_classes, stride=8),
    YOLOLayer(anchors[1], num_classes, stride=16),
    YOLOLayer(anchors[2], num_classes, stride=32)
])
```

- Mỗi YOLOLayer tạo ra một tensor đầu ra có dạng:

$$(B, 3, N, N, 5 + \text{num_classes})$$

Trong đó:

- B : batch size.
- 3 : số anchor box trên mỗi ô (cell).
- N : kích thước feature map (lần lượt là 52, 26, và 13).
- $5 + \text{num_classes}$: gồm 5 giá trị bounding box $(t_x, t_y, t_w, t_h, t_o)$ và xác suất lớp.



2.2 Output của anchor boxes

$$b_x = (c_x + \sigma(t_x)) \times \text{stride},$$

$$b_y = (c_y + \sigma(t_y)) \times \text{stride},$$

$$b_w = p_w \cdot e^{t_w},$$

$$b_h = p_h \cdot e^{t_h},$$

2.3 Why Sigmoid ?

- Giới hạn offset trong ô lưới: $\sigma(t_x), \sigma(t_y) \in [0, 1]$, đảm bảo tâm bounding box không vượt ra ngoài ô.
- Ổn định và dễ học: sigmoid giúp mạng học nhanh hơn vì giá trị dự đoán nằm trong khoảng có ý nghĩa, tránh lỗi lớn khi huấn luyện.

3 Loss function

3.1 YOLO-v2 Loss

$$\begin{aligned}\mathcal{L} = & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B 1_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ & + \sum_{i=0}^{S^2} 1_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2\end{aligned}$$

3.2 YOLO-v3 Loss - so với YOLO-v2

YOLOv3 có một số cải tiến so với YOLOv2 trong hàm loss, trong khi vẫn giữ cấu trúc tổng thể tương tự.

1. Bounding box loss:

YOLOv2 học trực tiếp x, y, w, h (với $\sqrt{w}, \sqrt{h}$ để ổn định):

$$\mathcal{L}_{\text{bbox}}^{\text{v2}} = \sum_{i,j} 1_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right]$$

YOLOv3 dự đoán các tọa độ biến đổi t_x, t_y, t_w, t_h :

$$x = \sigma(t_x) + c_x, \quad y = \sigma(t_y) + c_y, \quad w = p_w e^{t_w}, \quad h = p_h e^{t_h}$$

$$\mathcal{L}_{\text{bbox}}^{\text{v3}} = \sum_{i,j} 1_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right]$$

2. Confidence / objectness loss:

YOLOv2 sử dụng IoU làm target:

$$\mathcal{L}_{\text{conf}}^{\text{v2}} = \sum_{i,j} 1_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum_{i,j} 1_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2$$

YOLOv3 sử dụng target 0/1 với ignore mask cho các anchor gần đúng:

$$\mathcal{L}_{\text{obj}}^{\text{v3}} = \sum_{i,j} 1_{ij}^{\text{obj}} (\hat{C}_i - 1)^2 + \sum_{i,j} 1_{ij}^{\text{noobj}} (\hat{C}_i - 0)^2 \quad (\text{áp dụng ignore mask})$$

3. Class loss:

YOLOv2 sử dụng softmax và MSE (mỗi object chỉ có 1 class):

$$\mathcal{L}_{\text{class}}^{\text{v2}} = \sum_i 1_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

YOLOv3 sử dụng multi-label sigmoid và binary cross-entropy:

$$\mathcal{L}_{\text{class}}^{\text{v3}} = \sum_i 1_i^{\text{obj}} \sum_{c \in \text{classes}} \left[y_c \log \hat{p}_c + (1 - y_c) \log(1 - \hat{p}_c) \right]$$